

NAAN MUDHALAVAN

PROJECT REPORT

AI STUDY BUDDY API

A Study Assistance System Using Artificial Intelligence and REST API

Prepared for Academic Project Submission

Academic Year: 2026–2027

S.NO	TEAM	NAME	EMAIL ID
1	Team Leader	Nithya M	nithyammurugesan775@gmail.com
2	Team Member	Pavithara M.	pavithra809842@gmail.com
3	Team Member	Pavithara K	sridevipavi603@gmail.com
4	Team Member	Poojasri R	sripooja3960@gmail.com
5	Team Member	Premavathi M	kavithapremavathi@gmail.com

1. Abstract

AI Study Buddy API is a backend service designed to provide intelligent study assistance through a simple and reusable API. A client application can send a question or study-related request to the server, and the backend can process the request through an AI service and return a structured response.

The system uses a Node.js runtime with Express.js for HTTP routing and JSON-based communication. Environment variables are used for configuration and sensitive credentials. The API can support learning functions such as question answering, summarization, quiz generation and study assistance.

The API-first design separates the learning interface from backend logic. Therefore, the same backend can be connected to a web application, mobile application or another educational system.

2. Introduction

Students often need quick explanations, revision material and practice questions. Searching for suitable information across multiple resources can take time. An AI study assistant can provide an interactive learning layer that responds to natural-language questions.

An API, or Application Programming Interface, enables two software components to communicate. In this project, the client sends an HTTP request, the server validates and processes it, and a JSON response is returned.

3. Problem Statement

Traditional learning applications may depend heavily on static notes or predefined question banks. Such systems may not provide flexible, conversational assistance. The proposed API provides a common backend through which AI-powered study features can be accessed consistently.

4. Objectives

- Develop a backend API for AI-powered study assistance.
- Accept study questions and prompts through HTTP requests.
- Return clear and structured JSON responses.
- Provide reusable endpoints for web and mobile clients.
- Keep configuration and sensitive credentials outside source code.
- Validate input and handle errors safely.
- Use a modular architecture that supports future learning features.
- Provide a project structure suitable for academic demonstration and further development.

5. Scope of the Project

The scope includes the backend API, Express routes, request validation, AI-service communication, response formatting, error handling and development configuration. A frontend is not required for the backend to operate; any compatible client can consume the API.

Authentication, persistent chat history, analytics and advanced student management can be added later. The initial version concentrates on the core API and AI interaction.

6. Existing System and Proposed System

Existing learning systems commonly use static notes, search engines or fixed question banks. The proposed system adds a conversational AI layer where a student can submit a natural-language request and receive a generated study-oriented response.

The separation between frontend and backend also improves maintainability. Different client applications can use the same API without duplicating the AI integration.

7. System Requirements

Hardware Requirements

- Computer or laptop with at least 4 GB RAM.
- Modern processor capable of running Node.js.
- Stable internet connection when an external AI service is used.
- Storage space for project files and dependencies.

Software Requirements

- Windows, Linux or macOS.
- Node.js runtime and npm.
- Express.js framework.
- Visual Studio Code or another code editor.
- Postman or another REST API testing tool.
- .env-based configuration.
- Credentials for the selected AI provider, when applicable.

8. Technology Stack

Technology	Purpose
Node.js	JavaScript runtime for backend development
Express.js	HTTP server and routing framework
REST	API communication style
JSON	Request and response data format
.env	Environment-based configuration
AI Model/API	Generates study-oriented responses
Postman	API testing and verification

9. Functional Requirements

- API shall accept valid HTTP requests.
- API shall validate required input fields.
- Server shall process study prompts.
- Server shall communicate with the configured AI service.
- API shall return structured JSON.
- Invalid requests shall receive meaningful error responses.
- Configuration values shall be loaded from environment variables.

10. System Architecture

The system follows a layered backend architecture. The client sends a request to an Express route. The route validates the input and passes it to a controller or service. The AI service communicates with the selected AI provider and the result is formatted into a consistent JSON response.

Architecture Flow

```
Client Application → HTTP Request → Express Router → Validation / Controller → AI Service → Response Formatter → JSON Response → Client
```

This architecture reduces coupling. A frontend can change without rewriting the AI logic, and the AI provider can be changed by modifying the service layer.

11. Major Modules

11.1 Server Module

Starts the Node.js application, creates the Express application, loads configuration, registers middleware and starts listening on the configured port.

11.2 Route Module

Defines URLs and HTTP methods through which clients access the system. Routes should remain focused on receiving requests and passing work to appropriate controllers or services.

11.3 AI Service Module

Prepares the prompt or request payload and communicates with the external AI provider. Keeping this logic separate improves maintainability and testing.

11.4 Validation and Error Handling

Checks required fields and input format. Errors are converted into predictable HTTP responses so client applications can handle them correctly.

12. API Design

The API follows REST-style conventions. Requests and responses use JSON, and HTTP status codes communicate the result of an operation.

Method	Endpoint	Purpose
GET	/api/health	Check API/server status
POST	/api/study	Submit a study question or prompt
POST	/api/quiz	Generate practice questions
POST	/api/summarize	Summarize study text
GET	/api/info	Return API information

13. Sample Request and Response

Example request:

```
{ "question": "Explain database normalization in simple terms.", "level": "beginner" }
```

Example response:

```
{ "success": true, "answer": "Database normalization is a method of organizing data to reduce unnecessary duplication...", "topic": "DBMS" }
```

The exact response structure can be changed according to the final implementation. Consistent response fields make frontend integration easier.

14. HTTP Status Codes

- 200 OK — request completed successfully.
- 201 Created — resource created, where applicable.
- 400 Bad Request — missing or invalid input.
- 401 Unauthorized — authentication required or invalid.
- 404 Not Found — route or resource does not exist.
- 429 Too Many Requests — rate limit reached.
- 500 Internal Server Error — unexpected server-side problem.
- 502/503 — dependent AI service is unavailable.

15. Implementation Approach

The project can be created as a Node.js application with Express.js. The server initializes the application, enables JSON parsing and mounts API routes. Environment variables are loaded from a .env file. The controller handles request-level logic and the AI service handles communication with the configured model provider.

Suggested Project Structure

```
ai-study-buddy/  
├── server.js  
├── package.json  
├── .env  
├── routes/  
│   └── studyRoutes.js  
├── controllers/  
│   └── studyController.js  
├── services/  
│   └── aiService.js  
├── middleware/  
│   └── errorHandler.js  
└── README.md
```

16. Environment Configuration

Sensitive credentials should not be hard-coded. Typical configuration values include the server port and an AI provider key.

Example:

```
PORT=5000  
AI_API_KEY=your_api_key_here
```

The real secret value should remain private. The .env file should be excluded from version control using .gitignore.

17. Request Processing Flow

- Client sends a POST request containing a study prompt.
- Express receives and parses the JSON body.
- Validation checks required fields.
- Controller passes the prompt to the AI service.
- AI service sends the request to the configured provider.
- Generated content is received and normalized.
- API returns JSON to the client.
- Errors are caught and converted into controlled HTTP responses.

18. Testing

Testing verifies that the API behaves correctly for valid, invalid and failure conditions. Postman, curl or a frontend client can be used for manual testing.

Test Case 1 — Health Check

GET /api/health → Expected: HTTP 200 with a message showing that the API is running.

Test Case 2 — Valid Study Question

POST /api/study with a valid question → Expected: HTTP 200 and an answer field.

Test Case 3 — Missing Question

POST /api/study with an empty body → Expected: HTTP 400 with a validation message.

Test Case 4 — Invalid Route

GET /api/unknown → Expected: HTTP 404.

Test Case 5 — AI Service Failure

Simulate an unavailable AI provider → Expected: controlled error response without crashing the server.

19. Security Considerations

- Never expose AI API keys in frontend code.
- Store secrets in environment variables or a secure secret manager.
- Validate and limit user input.
- Use HTTPS in production.
- Add authentication when private student data is stored.
- Use rate limiting to protect public deployments.
- Do not return internal stack traces to clients.
- Avoid logging credentials or unnecessary private user content.

20. Performance Considerations

AI requests can take longer than ordinary API calls because the server may depend on an external model service. Timeouts, retries, rate limits and appropriate error handling should therefore be considered. Caching can help with repeated non-personalized requests, while streaming can improve the experience for long answers.

21. Advantages

- Reusable AI backend for web and mobile learning interfaces.
- Supports natural-language study interaction.
- Modular architecture is easier to maintain.
- JSON communication is simple and widely supported.
- Environment configuration improves credential management.
- New learning endpoints can be added incrementally.
- One backend can support multiple educational features.

22. Limitations

- AI-generated answers can occasionally be inaccurate and should be checked for important academic content.
- External AI services may require internet connectivity.
- Provider quotas or costs can limit usage.
- Public APIs require authentication and abuse protection.
- Response quality depends on prompt design and model capability.

23. Future Enhancements

- Student login and secure authentication.
- Conversation history and personalized profiles.
- Subject-wise study plans and reminders.
- Automatic quizzes with difficulty levels.
- Progress tracking and performance analytics.
- PDF/file-based question answering.
- Voice-based study interaction.
- Multilingual support including Tamil and English.
- Teacher/admin dashboard.
- Database integration for persistent learning history.

24. Deployment Overview

The application can be deployed to a Node.js-compatible cloud or server. Production deployment should use secure environment variables, HTTPS, logging, process monitoring and suitable rate limits. A deployed API URL can then be consumed by a web or mobile frontend.

25. Conclusion

The AI Study Buddy API provides a practical backend foundation for an intelligent learning assistant. Through REST-style endpoints, the system can connect study applications with AI capabilities while keeping the core processing logic centralized.

The project demonstrates Node.js backend development, Express routing, JSON communication, environment configuration, API validation, error handling and AI integration. It can serve as an academic project and can also be expanded into a complete learning platform.

A reliable implementation should emphasize clear API contracts, secure configuration, robust error handling and responsible use of AI-generated educational content. Authentication, study history, quizzes, analytics and multilingual assistance can be added in future versions.

26. References / Learning Resources

- Node.js documentation — runtime and server-side JavaScript concepts.
- Express.js documentation — routing, middleware and HTTP server development.
- REST API concepts — HTTP methods, status codes and JSON communication.
- Postman documentation — API testing.
- Documentation of the selected AI provider — authentication, requests and usage limits.

27. Viva / Presentation Points

- What is an API? — A software interface that allows applications to communicate.
- Why Node.js? — It provides a JavaScript runtime suitable for backend services.
- Why Express.js? — It simplifies routing, middleware and HTTP server development.
- Why JSON? — It is lightweight, readable and widely supported.
- Why use .env? — To keep configuration and secrets outside source code.
- What is the role of AI? — To generate study-oriented responses from user prompts.
- How is the API tested? — Using Postman and other REST testing methods.

— End of Documentation —